CRYPTOGRAPHY AND CAMERA HOMEBREW

Running code on ARM with lenses

Slides about crypto algorithms are TLP:AMBER

BIO

Laurent Clévy

- Forensic analyst in a SOC/CERT + some reverse engineering
- Following Magic Lantern (aka ML) activity since years
- Wanna be low level reverser
- Twitter: @lorenzo2472, Github: https://github.com/lclevy



Free time contributions

- Canon RAW v2 and RAW v3 file formats reference documentations
- Ported ML on 550d and 60d. Experiments on my R6
- Reversed Original Data Decision implementation from Canon in 2012, with python tool to recompute signatures. See this MISC article (open access): https://connect.ed-diamond.com/MISC/mischs-006/mecanisme-de-controle-d-authenticite-des-photographies-numeriques-dans-les-reflexes-canon
- Other MISC MAG articles

DIGITAL CAMS ARE COMPUTING DEVICES



Digital Single Lens Reflex (DSLR) or Mirrorless Interchangeable Lens (MILC) cameras are complex devices

- Multiples CPUs (main, AF, peripherals, GPU, face recognition, network ops, …)
- ARM-A9, ARM-M4 (mpu), Tensilica Xtensa (net), Takumi GV550 (gpu)
- RTOS (DryOS)
- Wifi, Bluetooth, Ethernet, GPS, USB, HDMI
- RAW image processing at 10-30 frame/sec



Solution : dedicated System on Chip for Canon, called DigIC (Digital IC)

So hackers want to run Doom on it !

https://www.youtube.com/watch?v=fAoIjXZYu70 (Doom on RP by @coon)

https://wiki.magiclantern.fm/digic (which CPUs per Digic generation)

STANDING ON GIANTS SHOULDERS

CHDK, Canon Hack DevKit (pocket cameras)

- https://chdk.fandom.com/wiki/CHDK
- Enhancing official firmware
  - RAW, LUA scripting,…
  - Loaded from sdcard, only in memory





Magic Lantern (DSLR)

- Created by Trammell Hudson (https://trmm.net/Magic_Lantern_firmware/)
  - Code execution in may 2009 on 5Dm2, with modified FIR update file
  - But blocked by signature check activation on 7D,
- 2010-2018 main contributors: A1ex (main dev), G3gg0 (reverse), Arm.Indy (crypto), …
- 2018-2022 : Kitor (reverse+dev), Names_are_hard, Coon, Petabyte, Turtius, …

FIR FILES (VERSION 4, 2007-2018)

FIR header
- Camera model
- Update version
- FIR file checksum (sum of bytes)
- Pointers to other sections (updater1, 2, 3, and fw records)
- Signature header (how to compute updaters signature)
- Updaters signature value (hmac-sha1)
- Firmware records signature value (hmac-sha1)

Updater1 header
- Encryption header
- Encrypted payload (AES)

…

Firmware header
- Encrypted header
- Encrypted payload (AES)

See
- https://groups.google.com/g/ml-devel/c/ASabsRbvgvQ/m/IKsMn3PPqPQJ?hl=en%7C7D (Trammell, 10/2009)
- https://magiclantern.fandom.com/wiki/Firmware_file (Indy, 2011)

```
---.fir header---
0x000: modelId = 0x80000250, (7D, DryOS)
0x010: version = 1.0.9
0x020: checksum = 0x9e5642e5                                    integrity
0x024: updater1 header = 0xb0
0x028: updater1 offset = 0x120
0x02c: updater2 offset = 0x1c0970
0x030: firmware offset = 0x22e200
0x034: 0xffffffff
0x038: embedded file size = 0xc41b8c
0x03c: 0x0
0x040: sha1 seed = 0xb99b53de
0x044: 0x00000004 0x00000000 0x00000020 0x00000024 0x00000044 0x000000b0 0x0022e150 0x22e200 0xa1398c
0x068: updaters hmac-sha1 = ec50923d17342689204568e42d13a427d5462234
0x088: firmware hmac-sha1 = a34761111161b5125c1dfd3e7f3348a77a92f956    authentication
---updater1 header---
0x0b0: updater1 length = 0x1c0850. starts at 0x120
0x0b4: 0x1c0850
0x0b8: 0x0
0x0bc: xor seed value = 0x6dcb1922
0x120: --- updater1 (ciphered) ---                              confidentiality
0x1c0970: --- updater1 end ---
---updater2 header---
0x1c0970: (+0x000), modelId = 0x80000250, (7D, DryOS)
0x1c0980: (+0x010), version = 1.0.9
0x1c0990: (+0x020), checksum = 0xfd568edf
0x1c0994: (+0x024), 0xb0
0x1c0998: (+0x028), 0x120
0x1c099c: (+0x02c), ffffffff ffffffff ffffffff
0x1c09a8: (+0x038), updater length (including header) = 0x6d890. starts at 0x1c0970
0x1c0a20: (+0x0b0), updater length = 0x6d770. starts at 0x1c0a90
0x1c0a24: (+0x0b4), 0x6d764
0x1c0a28: (+0x0b8), 0x0
0x1c0b4c: (+0x0bc), xor seed value = 0x789a414c
0x1c0a90: (+0x120), --- updater2 (ciphered) ---
---firmware header---
0x22e200: (+0x000), offset to decryption data? = 0xc
0x22e204: (+0x004), offset to encrypted data = 0x7c. starts at 0x22e200
0x22e208: (+0x008), total firmware length (including header) = 0xa1398c. starts at 0x22e200
-
0x22e20c: (+0x00c), firmware length (encrypted part) = 0xa13910. starts at 0x7c
0x22e210: (+0x010), sha1_in = 0x00a1390e
                                                                confidentiality
```

CODE EXECUTION : PATCHING UPDATER

40d (2007, VxWorks)

- Within update FIR file, updater code is only xored with 2 tables (512 and 513 bytes)
  - Updater is patched to dump memory (by ASalina, 2008)
  - Video hack based on liveview !
- Update records are AES encrypted

5dm2 (2008, DryOS)

- Bootflag activated by modified updater, then autoexec.bin loaded from sdcard
- Autoexec.bin : Canon feature likely for factory operations

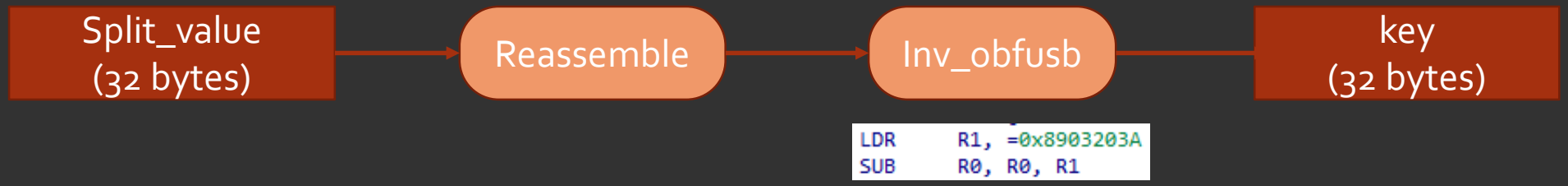
7d (2009)

- Signature check activated. FIR Signature and decryption reversed (2010)
- ML team is able to forge FIR files (update files) like Canon, but Crypto tools kept private

550d (2010)

- Updater is now encrypted with AES too, and keys are changed

V4, DECRYPT AND SIGN KEYS GENERATION



2007
EOS 40d

sign_key_2007

decrypt_key_2007

sign_key_2010

decrypt_key_2010

2010
EOS 550d

FIR V4: SIGNATURE GENERATION

TLP:AMBER

md5sums(value) == md5 (value + header.model_id (16 bytes)) + md5 (value + header.fir_version (16 bytes))

sha1sum(value) == sha1 (header.seed (4 bytes) + sign_key + value)

FIR file content

function

value

hmac_key (32 b) = sha1sum (b'\01') + sha1sum (b'\02')[0:12]

updaters_sign = hmac (hmac_key , md5sums (md5 (fir_header w/o chksum, signatures (using table at 0x44))), sha1)

firmware_sign = hmac (hmac_key , md5sums (md5 (firmware header + content)), sha1)

FIR V4: SIGNATURE GENERATION

TLP:AMBER

```
0x040: sign seed b'1ee2a9a4'
0x044: 0x4 (0x0, 0x20), (0x24, 0x44), (0xb0, 0x2f03e0), (0x2f0490, 0x1b5b410),
0x068: b'a63590206b32e9480b794333666d086fa95b0c81'
0x088: b'4424ef723e8950fe299690907eabeff9662ad1b2'
---updater1 header---
```

FIRv4 for 7d

Good

- Md5 for large amount of data (firmware, on 40D, 2007) because of slow CPU
- Include model and firmware version
- Same FIR format used since 2007, except signature fields. Nice design !

FIRv6 for R3

```
0x040: sign seed b'00000000'
0x044: 0x4 (0x0, 0x20), (0x24, 0x44), (0x100, 0x479550), (0x479650, 0x4bfd670),
0x068: 0x20 b'9c43abf0840d185143dfdf43fcaf8f03d0190dc97ae409e94b4e687946983304'
0x08c: 0x20 b'd6026381582fa9dff010ea35272ed65cae60157dadc65d0599fd6d8d2db37549'
0x0b0: 0x20 b'31e41512af7c71072b66e897e9f8e826100691a3757be4d84c5741ceeced4a4d'
0x0d4: 0x20 b'ad52fbe25b513b318afac3076c0e0b8a596e82a972ac22ca1fe31df99e417c68'
---updater1 header---
```

Strange/not good

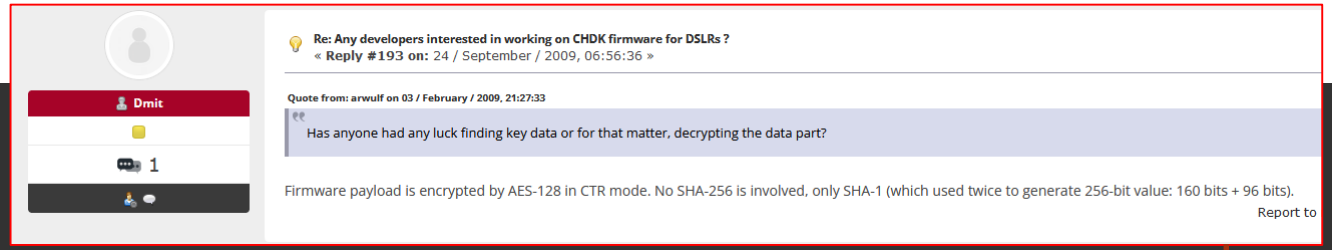
- Too complex: obfuscation ?
- Same keys for all models, only changed in 2010 (3 years) and 2018, despite Canon knowns ML team can forge valid FIR files since 2011

Note : FIRv5 since EOS R (digic8), FIRv6 since R5 (Digic X)

FIR V4: DECRYPTION

TLP:AMBER

```
---updater1 header---
0x0b0: encrypted length = 0x282d60
0x0b4: seed1 b'5c2d2800'
0x0b8: 0
0x0bc: seed2 b'uZ\xc2\xde'
0x0c0: iv1 b'ca597807a4eaa71e731a9b35145773ea'
0x0d0: aes_in b'4c28229c3d217552f90e37fd8949ecf8e150f426975aa296fcf8f4d992a61c15'
0x0f0: iv2 b'8503dd64715dd57cd5d8a13b89bfe00c'
0x100: check_val b'b6238beee58a9e33085864da64d875a5cbd0dfaf3326446aeb475f98c49e891f'
0x120: ---encrypted content---
```



Re: Any developers interested in working on CHDK firmware for DSLRs ?
« Reply #193 on: 24 / September / 2009, 06:56:36 »

Dmit

1

Quote from: arwulf on 03 / February / 2009, 21:27:33

Has anyone had any luck finding key data or for that matter, decrypting the data part?

Firmware payload is encrypted by AES-128 in CTR mode. No SHA-256 is involved, only SHA-1 (which used twice to generate 256-bit value: 160 bits + 96 bits).

Report to

check

stage1_key (16 b) = sha1 (seed2 + decrypt_key + seed1 + b'\01')[0:16]

aes_ctr1 (32b) : prefix = iv1 [0:12], init_value = iv1 [12:16]

stage1 (32b) = aes (k= stage1_key , mode=CTR, ctr= aes_ctr 1).encrypt(aes_in)

ref_value (32b) = sha1 (stage1 + b'\01') + sha1 (stage2 + b'\02')[0:12]

check_val ? == ref_value (32b)

decrypt

aes_ctr2(32b) : prefix = iv2 [0:12], init_value = iv2 [12:16]

cleartext = aes (k= stage1 [0:16], mode=CTR, ctr= aes_ctr 2).encrypt(content)

DEMO : DECRYPTING AND SIGNING V4

TLP:AMBER

```
---.fir header---
0x000: modelId = 0x80000349
0x010: version = b'1.3.2'
0x020: checksum = 0xf5a2d70
0x024: updater1 header = 0xb0
0x028: updater1 offset = 0x120
0x02c: updater2 offset = 0xffffffff
0x030: firmware offset = 0x2f0490
0x034: updater3 offset = 0x282e80
0x038: filesize = 0x1e4b8a0
0x03c: 0x0
0x040: sign seed b'1ee2a9a4'
0x044: 0x4 (0x0, 0x20), (0x24, 0x44), (0xb0, 0x2f03e0), (0x2f0490, 0x1b5b410),
0x068: b'a63590206b32e9480b794333666d086fa95b0c81'
0x088: b'4424ef723e8950fe299690907eabeff9662ad1b2'
---updater1 header---
0x0b0: encrypted length = 0x282d60
0x0b4: seed1 b'5c2d2800'
0x0b8: 0
0x0bc: seed2 b'uZ\xc2\xde'
0x0c0: iv1 b'ca597807a4eaa71e731a9b35145773ea'
0x0d0: aes_in b'4c28229c3d217552f90e37fd8949ecf8e150f426975aa296fcf8f4d992a61c15'
0x0f0: iv2 b'8503dd64715dd57cd5d8a13b89bfe00c'
0x100: check_val b'b6238beee58a9e33085864da64d875a5cbd0dfaf3326446aeb475f98c49e891f'
0x120: ---encrypted content---
---updater3---
0x282e80: modelId = 0x80000349
0x282e90: version = b'1.3.2'
0x282ea0: checksum = 0xfd5c795a
0x282ea4: updater1 header = 0xb0
0x282ea8: updater1 offset = 0x120
```

5Dm4, version 1.3.2

updater1
and updater3 embedded

```
sign OK
writing 0349_132_updater1.bin
writing 0349_132_updater3.bin
writing 0349_132_firmware.bin
decrypted patches checksum correct? True
      + tag  + foffset  +   size   + moffset  +
 ----------------------------------------------
 0x01: 0x0200 0x00000170 0x00000e49 0x00000000
 0x02: 0x0200 0x00000fc0 0x000b6ef4 0x00000000
 0x03: 0x0200 0x000b7eb8 0x000bfb4a 0x00000000
 0x04: 0x0200 0x00177a08 0x0002649c 0x00000000
 0x05: 0x0200 0x0019dea8 0x00146f43 0x00000000
 0x06: 0x0100 0x002e4df0 0x0003eddc 0xfc000000
 0x07: 0x0100 0x00323bd0 0x001173b4 0x00010000
 0x08: 0x0100 0x0043af88 0x00000014 0x00000000
 0x09: 0x0100 0x0043afa0 0x005d8efc 0xfd980000
 0x0a: 0x0100 0x00a13ea0 0x00eb2304 0xfc0a0000
 0x0b: 0x0102 0x018c61a8 0x000001a6 0x00000000
 0x0c: 0x010a 0x018c6350 0x0003e7d4 0xfc000000
 0x0d: 0x010a 0x01904b28 0x00002d50 0xfca20000
 0x0e: 0x010a 0x01907878 0x00253b0c 0xfc0a0000
```

Signature verification

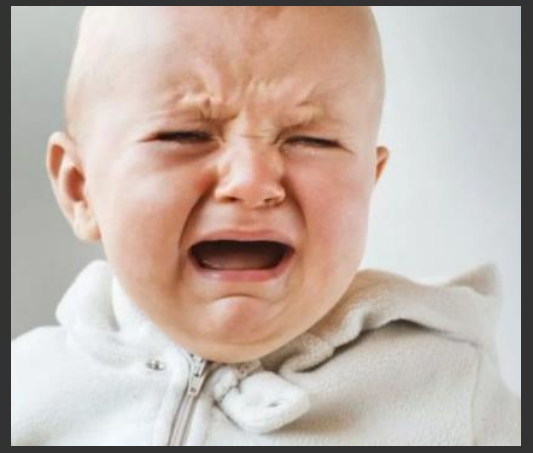
Decrypted code + data

type 0x200 is for mpu

Example:
record#10,
type 0X100 (main CPU),
size 0XA13EA0,
written at 0xFC0A0000

EOS R (09/2018) : CRYPTO CHANGED

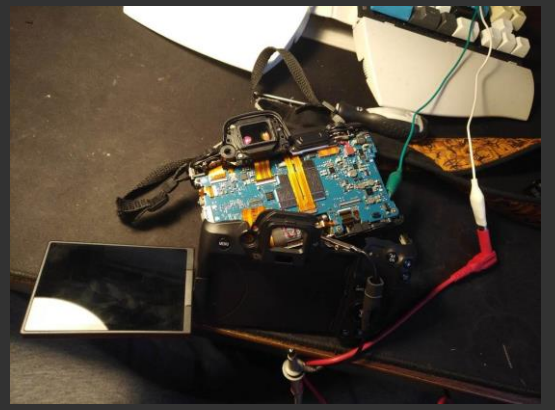
- FIR encryption changed, ML team can not decrypt Canon updates and reverse them for porting code
- FIR signature changed, no way to forge valid updates, run code to dump ROM
- No dumps, no reverse, no hacking !



EOS R, UART ACCESS

14/02/2019

- Kitor activated Bootflag via UART
- Custom autoexec.bin payload was used to dump
- Not suited to allow everyone to dump and port ML ☹





2/09/2020

- Free gift: Canon basic is available on EOS R !
- Discovered by Oren Isacson Alfredo Ortega, presented at DEFCON 18 (07/2010)
  - https://www.youtube.com/watch?v=ByuHcamy2Z4
- Known by CHDK people since 2010
  - https://chdk.setepontos.com/index.php/topic,5549.0.html
- Was not available on previous DSLRs, except m50 (04/2018)



CANON BASIC

Scripts can be executed under specific conditions

- See https://chdk.fandom.com/wiki/Canon_Basic

Format of the SD card

- The SD card may be in FAT16 or FAT32 format.
- The SD card must contain the following items:

1. The string "SCRIPT" must be at offset 0x1F0 of the first sector (Boot sector).
2. The file "script.req" must exist on the card's root directory, and must only contain the string "for DC_scriptdisk\n" (where the \n represents a newline character)
3. The file "extend.m" or "autotest.m" must exist on the root directory. This file must contain the Canon BASIC script to execute.

Scripts examples:

- Dump main ROM

```
private sub Initialize()
    SaveAllRomImageToFile()
end sub
```

- Enable Bootflag

```
EnableBootDisk()
```

- Dump camera log

```
dumpf()
```

```
buf = umalloc(0x1000)
if buf=0 then
    exit sub
end if
bufo = umalloc(0x1000)
if bufo=0 then
    ufree(buf)
    exit sub
end if
memset(bufo,0,0x1000)
f = OpenFileRD(cpuinf_binary)
FIO_ReadFile(f,buf,0x1000)
CloseFile(f)
ExportToEventProcedure("cpuinfo",buf|1)
cpuinfo(bufo)
```

- Allocate memory, load ARM code and jump to it with parameter !
  - See https://github.com/lclevy/cbasic_examples/blob/main/R6.150/extend_cpuinfo.m

P TP CANON EVENT PROCEDURES

Event procedures are "call by name" functions, works with PTP

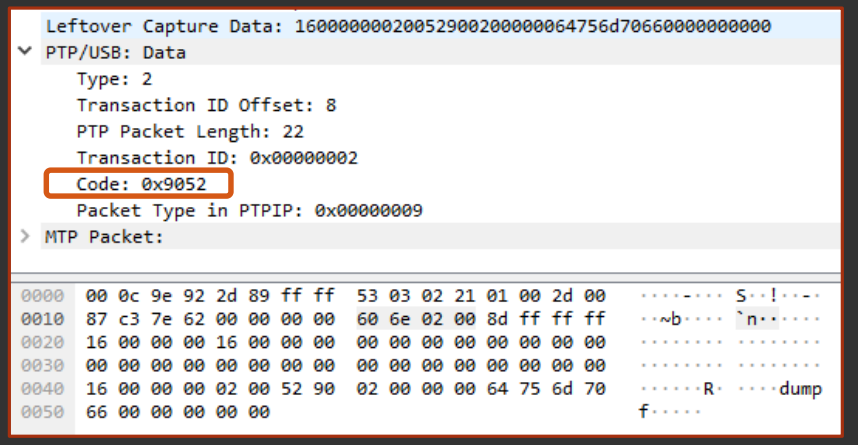
- Available from Canon Basic
- From DryOS, ARM code: call('enablebootdisk');
- From Picture Transfer Protocol, via a custom command : 0x9052

Petabyte (June 2021)

- Enabling bootflag using PTP on 1300D (digic4+)
- Reference client implementation is Canon EDSDK.DLL (from EOS UTILITY)

SRSA (06/02/2022)

- Describe EventProcedure call encoding with parameters
- See https://chdk.setepontos.com/index.php?topic=4338.msg147738#msg147738



CODE EXEC PATH PER DIGIC GENERATIONS

Code exec method	40D 2007 (d3)	5Dm2 2008 (d4)	1300D 2016 (d4+)	5D3 2012 (d5)	80D 2016 (d6)	5Dm4 2016 (d6+)	6D2 2017 (d7)	M50 04/2018 (d8)	EOS R 09/2018 (d8)	R6 2020 (dx)	R10 2022 (dx)
Cbasic	no	no	no	no	no	no	no	ok	ok	ok	locked
Fir v4	yes	yes	yes	yes	yes	yes	yes	yes	no, v5	no, v6	?
autoexec	?	yes	yes	yes	yes	yes	yes	yes	yes	yes	?
ptp	no?	yes	yes	yes	yes	yes	yes	yes	locked	locked	locked
uart	no?	?	ok	ok	ok	yes	ok	ok	ok	ok	locked

Canon needs code execution in factories to personalize / calibrate cameras

- Cbasic, autoexec and PTP features do certainly exist for that

Imagination (and time) are the limits …

OTHER REFERENCES

EOS 300d decryption, Alex Berstein, 2003
cpu=x86, mpu=mips, os=DOS
https://web.archive.org/web/20100814164946/wiki.alexbernstein.com/FirmwareDecrypterUnpacker

Homebrew firmware for the 5D Mark 2, Trammell Hudson (17/05/2009)
https://www.dvinfo.net/forum/canon-eos-full-frame-hd/235535-homebrew-firmware-5d-mark-2-a.html

ML running on 550d , Trammell Hudson (31/07/2010)
https://www.dvinfo.net/forum/canon-eos-crop-sensor-hd/482742-magic-lantern-demo-550d.html

Magic Lantern Free Software on your camera, Libre Graphics Meeting, Leipzip, 2/04/2014,
https://libregraphicsmeeting.org/2014/slides/magic.lantern.libgre.graphics.meet.2014.pdf
https://download.gimp.org/pub/gimp/lgm/2014/day_1/017_Magic_Lantern__Free_Software_on_Your_Camera/017_Magic_Lantern__Free_Software_on_Your_Camera.ogv

EOS firmware in QEMU - development and reverse engineering guide, A1ex
https://foss.heptapod.net/magic-lantern/magic-lantern/-/blob/branch/qemu/contrib/qemu/HACKING.rst

ML discord server : https://discord.gg/TyRPtvAS